

Introduction to Machine Learning

Foundations, Data Preparation, EDA and Validation

Dr. Puneet Bawa

School of Computer Science and Engineering, IILM University Gurugram

Academic Session 2026–27

Course coverage

- S1: AI, Data Science and ML
- S2: Evolution and applications of ML
- S3: Types of ML
- S4: ML workflow
- S5: Data types and data collection
- S6: Preprocessing and cleaning
- S7: Missing values and outliers
- S8: Feature engineering and selection
- S9: Normalization and standardization
- S10: Exploratory Data Analysis
- S11: Evaluation and validation
- S12: Training and testing datasets

Artificial Intelligence, Data Science and Machine Learning

Area		Meaning
Artificial Intelligence	Intelli-	Building systems that perform tasks associated with human intelligence, such as reasoning, perception and decision-making.
Data Science		Extracting useful knowledge from data through statistics, programming, domain knowledge and communication.
Machine Learning		A branch of AI where systems learn patterns from data to make predictions or decisions.

Relationship

Machine Learning is a subset of AI. Data Science often uses ML, but also includes data collection, analysis and visualisation.

Everyday examples

Problem	Input data	Output
Email filtering	words, sender, links	Spam or Not Spam
Student support	attendance, tests, study hours	Pass/Fail risk
Online recommendation	previous views and purchases	Suggested products
Speech recognition	audio signal	Recognised text

Evolution of Machine Learning

- **1950s–1960s:** early AI, perceptrons and pattern recognition.
- **1970s–1980s:** expert systems and rule-based decision-making.
- **1990s–2000s:** statistical learning, support vector machines and ensemble methods.
- **2010s:** big data, GPUs and deep learning.
- **Today:** foundation models, generative AI, responsible AI and domain-specific intelligent systems.

Key reason for progress

More data, stronger computing resources, improved algorithms and open-source software accelerated ML adoption.

Applications of Machine Learning

- Healthcare: diagnosis support and risk prediction
- Finance: fraud detection and credit scoring
- Education: learning analytics and early alerts
- Agriculture: crop disease detection
- Cybersecurity: anomaly and malware detection
- Retail: recommendation and demand forecasting
- Transport: route optimisation and autonomous systems
- Language: translation, speech recognition and chatbots

Supervised Learning

Supervised learning uses labelled examples: every training record has input features X and a known target y .

$$X \rightarrow y$$

Task	Target	Example
Classification	Category or class	Predict Pass/Fail
Regression	Continuous number	Predict final marks

Examples: Linear Regression, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes.

Unsupervised and Reinforcement Learning

Type	Main idea	Example
Unsupervised Learning	Finds structure in unlabelled data	Group customers with similar behaviour
Reinforcement Learning	An agent learns actions by rewards and penalties	Robot navigation or game playing

Unsupervised tasks

Clustering, dimensionality reduction and association-rule discovery.

Reinforcement-learning terms

Agent, environment, action, reward, policy and episode.

Machine Learning workflow

- ① Define the problem and success metric.
- ② Collect and understand data.
- ③ Clean, transform and prepare features.
- ④ Split data into training and testing subsets.
- ⑤ Train candidate models.
- ⑥ Evaluate models on unseen data.
- ⑦ Deploy, monitor and update when required.

Important

The process is iterative. Evaluation findings may require changes to data, features or the problem definition.

Problem formulation: Student Performance dataset

Goal: prepare data for a binary classification task.

Target : pass_fail (0 = Fail, 1 = Pass)

Feature category	Examples from the notebook
Numerical	age, test scores, attendance, study hours, assignments submitted
Categorical	gender, class, subject
Identifiers to exclude	student_id, name, teacher_id
Potential leakage variable	final_grade_letter

Types of data

Type	Description	Examples
Numerical	Measurable quantities	score, age, attendance percentage
Categorical	Labels or groups	subject, gender, class
Ordinal	Ordered categories	low, medium, high; letter grades
Text	Natural language content	feedback comments, emails
Time-series	Observations ordered by time	daily sales, sensor readings
Image/Audio/Video	Unstructured multimedia data	X-rays, speech, CCTV videos

Data collection techniques

- Surveys, forms and questionnaires
- Interviews and observations
- Sensors, IoT devices and system logs
- Transactional databases and ERP/LMS records
- APIs, web data and public repositories
- Experiments and controlled studies

Quality questions

Is the data relevant, accurate, complete, representative, timely and collected ethically with appropriate consent?

Why data preprocessing matters

Raw data may contain missing values, duplicates, inconsistent categories, extreme values and unsuitable formats.

- Many algorithms cannot accept missing values directly.
- Categorical values must often be converted to numbers.
- Different numerical scales can distort distance-based models.
- Leakage can make a model look better than it really is.

Preprocessing goal

Create a clean, meaningful and leakage-safe feature matrix for modelling.

Notebook: import and inspect data

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, MinMaxScaler

df = pd.read_csv("student_performance.csv")
display(df.head(10))
print("Dataset shape:", df.shape)
df.info()
```

Useful inspection commands

`head()`, `tail()`, `shape`, `columns`, `dtypes`, `info()` and `describe()` provide an initial understanding of a dataset.

Checking missing values and duplicates

```
missing_count = df.isnull().sum()
missing_percent = (missing_count / len(df)) * 100

missing_table = pd.DataFrame({
    "Missing Values": missing_count,
    "Missing Percentage": missing_percent.round(2)
})
display(missing_table)

print("Duplicate rows:", df.duplicated().sum())
df = df.drop_duplicates().reset_index(drop=True)
```

Data cleaning decisions

Issue	Possible response
Duplicate records	Remove exact duplicates after confirming they are not valid repeated events.
Inconsistent labels	Standardise spelling, case and coding, such as 'Male', 'male' and 'M'.
Wrong data type	Convert a number stored as text to numeric after checking invalid values.
Identifier fields	Usually exclude from predictive features.
Target leakage	Exclude variables unavailable at the time of prediction.

Handling missing values

Method	Suitable for	Caution
Drop rows/columns	Few missing records or unusable feature	Can discard useful information
Mean imputation	Roughly symmetric numeric data	Sensitive to extreme values
Median imputation	Skewed numeric data or outliers	Does not preserve full variability
Mode imputation	Categorical features	Can over-represent common category
Model-based methods	Complex missingness	Requires careful validation

Notebook: training-only imputation

```
numeric_imputer = SimpleImputer(strategy="mean")
X_train_processed[num_features] = numeric_imputer.fit_transform(
    X_train_processed[num_features])
X_test_processed[num_features] = numeric_imputer.transform(
    X_test_processed[num_features])

categorical_imputer = SimpleImputer(strategy="most_frequent")
X_train_processed[cat_features] = categorical_imputer.fit_transform(
    X_train_processed[cat_features])
X_test_processed[cat_features] = categorical_imputer.transform(
    X_test_processed[cat_features])
```

Rule

Use `fit_transform()` on training data and only `transform()` on testing data.

Outliers and the IQR method

$$IQR = Q_3 - Q_1$$

Possible outlier thresholds:

$$Lower = Q_1 - 1.5(IQR), \quad Upper = Q_3 + 1.5(IQR)$$

Values outside these limits are possible outliers.

Do not delete automatically

An outlier may be an entry error, a rare but valid observation, or an important event. Investigate its cause before removal or transformation.

Notebook: IQR outlier detection

```
q1 = df[column].quantile(0.25)
q3 = df[column].quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr

outliers = df[(df[column] < lower_bound) |
               (df[column] > upper_bound)]
print("Possible outliers:", len(outliers))
```

Feature engineering

Feature engineering creates or transforms variables so that the model can use meaningful patterns.

Notebook example: average score for EDA

```
df_eda["average_score"] = df_eda[  
    ["test1_score", "test2_score", "final_exam_score"]  
].mean(axis=1)
```

Potential engineered features include ratios, counts, dates decomposed into parts and domain-relevant combinations.

Feature selection and leakage

Feature selection keeps useful predictors and removes irrelevant, redundant or harmful ones.

- Filter methods: correlation, variance and statistical tests
- Wrapper methods: recursive feature elimination
- Embedded methods: L1 regularisation and tree-based importance

Target leakage example

If `final_grade_letter` comes from final performance, it may directly reveal `pass_fail`. Do not use it when predicting a student's outcome before results are known.

Notebook: separate features and target

```
target_column = "pass_fail"
columns_to_drop = [
    "student_id", "name", "teacher_id", "final_grade_letter"
]

X = df.drop(columns=columns_to_drop + [target_column])
y = df[target_column]

num_features = X.select_dtypes(include=np.number).columns.tolist()
cat_features = X.select_dtypes(exclude=np.number).columns.tolist()
```

Standardization and normalization

Technique	Formula	Typical need
Standardization	$z = \frac{x - \mu}{\sigma}$	KNN, SVM, Logistic Regression
Min-max normalization	$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$	Features desired in 0–1 range

Important

Decision Trees and Random Forests usually do not require scaling, but distance- and gradient-based methods often do.

Notebook: standardization and normalisation

```
scaler = StandardScaler()
X_train_standardized[num_features] = scaler.fit_transform(
    X_train_encoded[num_features])
X_test_standardized[num_features] = scaler.transform(
    X_test_encoded[num_features])

minmax_scaler = MinMaxScaler()
X_train_normalized[num_features] = minmax_scaler.fit_transform(
    X_train_encoded[num_features])
X_test_normalized[num_features] = minmax_scaler.transform(
    X_test_encoded[num_features])
```

Choose one scaling method for a basic workflow. Do not apply both to the same feature without a specific reason.

Exploratory Data Analysis

EDA examines data before modelling to understand distributions, patterns, relationships and data-quality concerns.

Analysis	Question	Common tools
Univariate	What does one variable look like?	histogram, box-plot, frequency table
Bivariate	How do two variables relate?	scatter plot, grouped mean, bar chart
Multivariate	How do many variables relate?	correlation matrix, heatmap, cross-tabulation

Notebook: univariate EDA

```
# Histogram for a numerical variable
plt.hist(df[column].dropna(), bins=20, edgecolor="black")
plt.title(f"Distribution of {column}")
plt.show()

# Frequency table for a categorical variable
frequency = df["subject"].value_counts(dropna=False)
percentage = df["subject"].value_counts(
    dropna=False, normalize=True).mul(100).round(2)
display(pd.DataFrame({"Count": frequency,
                      "Percentage": percentage}))
```

Notebook: bivariate EDA

```
plt.scatter(df["study_hours_per_week"],
            df["final_exam_score"], alpha=0.6)
plt.xlabel("Study Hours per Week")
plt.ylabel("Final Exam Score")
plt.show()

subject_exam = (df.groupby("subject")["final_exam_score"]
                 .mean().sort_values(ascending=False))
subject_exam.plot(kind="bar")
plt.show()
```

Interpret scatter plots cautiously: a pattern suggests association, not causation.

Notebook: correlation analysis

```
correlation_columns = analysis_numeric_columns + ["pass_fail"]
correlation_matrix = df[correlation_columns].corr()
display(correlation_matrix.round(3))

target_correlations = (
    correlation_matrix["pass_fail"].drop("pass_fail")
    .sort_values(key=abs, ascending=False)
)
display(target_correlations)
```

Correlation ranges from approximately -1 to $+1$. It describes linear association and does not establish causation.

Classification evaluation: confusion matrix

	Predicted Pass	Predicted Fail
Actual Pass	True Positive (TP)	False Negative (FN)
Actual Fail	False Positive (FP)	True Negative (TN)

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Regression evaluation and validation

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|, \quad RMSE = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

- Use a validation set or cross-validation to choose hyperparameters.
- Use the test set only for the final unbiased estimate.
- For imbalanced classification, accuracy alone can be misleading. Report precision, recall and F1-score.

Training and testing datasets

Subset	Role
Training set	Fits model parameters and preprocessing statistics.
Validation set / cross-validation	Chooses settings such as K , tree depth or regularisation.
Testing set	Evaluates final performance on data not used during learning or tuning.

Common split

An 80:20 train–test split is a practical starting point for many datasets.

Notebook: stratified 80:20 split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.20,  
    random_state=42,  
    stratify=y  
)  
  
print("X_train:", X_train.shape)  
print("X_test :", X_test.shape)  
display(y_train.value_counts(normalize=True).mul(100))  
display(y_test.value_counts(normalize=True).mul(100))
```

stratify=y preserves the approximate Pass/Fail proportion in both subsets.
random_state=42 makes the split reproducible.

Data leakage

Data leakage occurs when information from the test set or the future affects model training.

- Scaling before splitting uses test-set mean and standard deviation.
- Imputing before splitting uses test-set missing-value information.
- Selecting features using the whole dataset can inflate apparent performance.
- Using final marks for an early-warning prediction is leakage if final marks are unavailable at prediction time.

Correct sequence

Split first. Fit imputer, encoder, scaler and feature-selection steps on training data only. Apply learned transformations to test data.

Recommended preprocessing pipeline

```
numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore", drop="first"))
])
preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, num_features),
    ("categorical", categorical_pipeline, cat_features)
])
X_train_ready = preprocessor.fit_transform(X_train)
X_test_ready = preprocessor.transform(X_test)
```

Key revision points

- Define the target and prediction time before selecting features.
- Inspect data types, missingness, duplicates and class balance before modelling.
- Use EDA to understand distributions and relationships.
- Treat IQR flags as possible outliers, not automatic deletion decisions.
- Encode categories and scale features when the algorithm requires it.
- Separate training and testing data before fitting preprocessing steps.
- Evaluate classification and regression using task-appropriate metrics.

Practice and viva questions

- ➊ Why should identifiers such as student ID and name usually be excluded from a model?
- ➋ When is median imputation preferred over mean imputation?
- ➌ What does `stratify=y` achieve in a train–test split?
- ➍ Why does correlation not imply causation?
- ➎ Which metric matters more when missing a positive case is costly: precision or recall?
- ➏ Explain one example of target leakage in the Student Performance dataset.

Thank You

Questions and Discussion